Python Interview Programs — With Built-ins and Without Built-ins

---

Strings

1. Reverse a String

Using Built-ins

```
s = "hello"
print(s[::-1])
```

Without Built-ins

```
s = "hello"
rev = ""

for ch in s:
    rev = ch + rev

print(rev)
```

---

2. Check Palindrome

Using Built-ins

```
s = "madam"
print(s == s[::-1])
```

Without Built-ins

```
s = "madam"
left = 0
right = len(s) - 1
is_palindrome = True
```

```
while left < right:
    if s[left] != s[right]:
        is_palindrome = False
        break
    left += 1
    right -= 1

print(is_palindrome)
```

---

3. Count Vowels and Consonants

Using Built-ins

```
s = "interview"
vowels = "aeiouAEIOU"

v = sum(1 for ch in s if ch in vowels)
c = sum(1 for ch in s if ch.isalpha() and ch not in vowels)

print(v, c)
```

Without Built-ins

```
s = "interview"
vowels = "aeiouAEIOU"
v = 0
c = 0

for ch in s:
    if ('a' <= ch <= 'z') or ('A' <= ch <= 'Z'):
        found = False
        for x in vowels:
            if ch == x:
                found = True
                break

        if found:
            v += 1
        else:
            c += 1

print(v, c)
```

4. Find First Non-Repeating Character

Using Built-ins

```
from collections import Counter

s = "swiss"
count = Counter(s)

for ch in s:
    if count[ch] == 1:
        print(ch)
        break
```

Without Built-ins

```
s = "swiss"

for i in range(len(s)):
    count = 0

    for j in range(len(s)):
        if s[i] == s[j]:
            count += 1

    if count == 1:
        print(s[i])
        break
```

---

5. Check Anagram

Using Built-ins

```
s1 = "listen"
s2 = "silent"

print(sorted(s1) == sorted(s2))
```

Without Built-ins

```
s1 = "listen"
s2 = "silent"

if len(s1) != len(s2):
    print(False)
else:
    freq = {}

    for ch in s1:
        freq[ch] = freq.get(ch, 0) + 1

    for ch in s2:
        if ch not in freq:
            print(False)
            break
        freq[ch] -= 1

    result = True
    for val in freq.values():
        if val != 0:
            result = False

    print(result)
```

---

6. Remove Duplicates from String

Using Built-ins

```
s = "programming"
print("".join(dict.fromkeys(s)))
```

Without Built-ins

```
s = "programming"
result = ""

for ch in s:
    exists = False

    for x in result:
        if x == ch:
```

```
            exists = True
            break

    if not exists:
        result += ch

print(result)
```

---

7. Count Frequency of Each Character

Using Built-ins

```
from collections import Counter

s = "hello"
print(Counter(s))
```

Without Built-ins

```
s = "hello"
freq = {}

for ch in s:
    if ch in freq:
        freq[ch] += 1
    else:
        freq[ch] = 1

print(freq)
```

---

8. Longest Substring Without Repeating Characters

Using Built-ins

```
s = "abcabcbb"
seen = set()
left = 0
max_len = 0

for right in range(len(s)):
```

```
    while s[right] in seen:
        seen.remove(s[left])
        left += 1

    seen.add(s[right])
    max_len = max(max_len, right - left + 1)

print(max_len)
```

Without Built-ins

```
s = "abcabcbb"
max_len = 0

for i in range(len(s)):
    temp = ""

    for j in range(i, len(s)):
        repeated = False

        for ch in temp:
            if ch == s[j]:
                repeated = True
                break

        if repeated:
            break

        temp += s[j]

    if len(temp) > max_len:
        max_len = len(temp)

print(max_len)
```

Lists / Arrays

1. Find Largest and Second Largest Number

Using Built-ins

```
arr = [10, 20, 4, 45, 99]
arr.sort()
print(arr[-1], arr[-2])
```

Without Built-ins

```
arr = [10, 20, 4, 45, 99]
first = second = float('-inf')

for num in arr:
    if num > first:
        second = first
        first = num
    elif num > second and num != first:
        second = num

print(first, second)
```

---

2. Reverse a List Without Built-ins

Using Built-ins

```
arr = [1, 2, 3, 4]
print(arr[::-1])
```

Without Built-ins

```
arr = [1, 2, 3, 4]
left = 0
right = len(arr) - 1

while left < right:
    arr[left], arr[right] = arr[right], arr[left]
    left += 1
```

```
    right -= 1

print(arr)
```

---

3. Remove Duplicates from List

Using Built-ins

```
arr = [1, 2, 2, 3, 4, 4]
print(list(set(arr)))
```

Without Built-ins

```
arr = [1, 2, 2, 3, 4, 4]
result = []

for num in arr:
    if num not in result:
        result.append(num)

print(result)
```

---

4. Find Missing Number in Array

Using Built-ins

```
arr = [1, 2, 4, 5]
n = 5

expected = n * (n + 1) // 2
print(expected - sum(arr))
```

Without Built-ins

```
arr = [1, 2, 4, 5]
n = 5

expected = 0
for i in range(1, n + 1):
```

```
    expected += i

actual = 0
for num in arr:
    actual += num

print(expected - actual)
```

---

5. Find Duplicate Number

Using Built-ins

```
arr = [1, 3, 4, 2, 2]
seen = set()

for num in arr:
    if num in seen:
        print(num)
        break
    seen.add(num)
```

Without Built-ins

```
arr = [1, 3, 4, 2, 2]

for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
        if arr[i] == arr[j]:
            print(arr[i])
```

---

6. Move Zeros to End

Using Built-ins

```
arr = [0, 1, 0, 3, 12]
non_zero = [x for x in arr if x != 0]
zeros = [0] * (len(arr) - len(non_zero))

print(non_zero + zeros)
```

Without Built-ins

```
arr = [0, 1, 0, 3, 12]
index = 0

for i in range(len(arr)):
    if arr[i] != 0:
        arr[index] = arr[i]
        index += 1

while index < len(arr):
    arr[index] = 0
    index += 1

print(arr)
```

---

7. Two Sum Problem

Using Built-ins

```
arr = [2, 7, 11, 15]
target = 9
lookup = {}

for i, num in enumerate(arr):
    diff = target - num

    if diff in lookup:
        print(lookup[diff], i)

    lookup[num] = i
```

Without Built-ins

```
arr = [2, 7, 11, 15]
target = 9

for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
        if arr[i] + arr[j] == target:
            print(i, j)
```

---

8. Merge Two Sorted Lists

Using Built-ins

```
a = [1, 3, 5]
b = [2, 4, 6]

print(sorted(a + b))
```

Without Built-ins

```
a = [1, 3, 5]
b = [2, 4, 6]

result = []
i = j = 0

while i < len(a) and j < len(b):
    if a[i] < b[j]:
        result.append(a[i])
        i += 1
    else:
        result.append(b[j])
        j += 1

while i < len(a):
    result.append(a[i])
    i += 1

while j < len(b):
    result.append(b[j])
    j += 1

print(result)
```

---

9. Rotate Array/List

Using Built-ins

```
arr = [1, 2, 3, 4, 5]
k = 2
```

```
print(arr[-k:] + arr[:-k])
```

Without Built-ins

```
arr = [1, 2, 3, 4, 5]
k = 2
n = len(arr)
result = [0] * n

for i in range(n):
    result[(i + k) % n] = arr[i]

print(result)
```

---

10. Find Common Elements in Two Lists

Using Built-ins

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]

print(list(set(a) & set(b)))
```

Without Built-ins

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]
result = []

for x in a:
    for y in b:
        if x == y and x not in result:
            result.append(x)

print(result)
```

Dictionaries / Hashing

1. Count Word Frequency in a Sentence

Using Built-ins

```
from collections import Counter

sentence = "python is fun python is easy"
words = sentence.split()

print(Counter(words))
```

Without Built-ins

```
sentence = "python is fun python is easy"
words = sentence.split()
freq = {}

for word in words:
    if word in freq:
        freq[word] += 1
    else:
        freq[word] = 1

print(freq)
```

---

2. Group Anagrams

Using Built-ins

```
from collections import defaultdict

words = ["eat", "tea", "tan", "ate", "nat", "bat"]
groups = defaultdict(list)

for word in words:
    key = ''.join(sorted(word))
    groups[key].append(word)

print(list(groups.values()))
```

Without Built-ins

```
words = ["eat", "tea", "tan", "ate", "nat", "bat"]
result = {}

for word in words:
    chars = list(word)

    for i in range(len(chars)):
        for j in range(i + 1, len(chars)):
            if chars[i] > chars[j]:
                chars[i], chars[j] = chars[j], chars[i]

    key = ''
    for ch in chars:
        key += ch

    if key not in result:
        result[key] = []

    result[key].append(word)

print(list(result.values()))
```

---

3. Find Most Frequent Element

Using Built-ins

```
from collections import Counter

arr = [1, 3, 2, 1, 4, 1]
print(Counter(arr).most_common(1))
```

Without Built-ins

```
arr = [1, 3, 2, 1, 4, 1]
freq = {}

for num in arr:
    freq[num] = freq.get(num, 0) + 1

max_count = 0
result = None
```

```
for key in freq:
    if freq[key] > max_count:
        max_count = freq[key]
        result = key

print(result)
```

---

4. Check if Two Strings are Isomorphic

Using Built-ins

```
s = "egg"
t = "add"

map1 = {}
map2 = {}
result = True

for a, b in zip(s, t):
    if map1.get(a, b) != b or map2.get(b, a) != a:
        result = False
        break

    map1[a] = b
    map2[b] = a

print(result)
```

Without Built-ins

```
s = "egg"
t = "add"

m1 = {}
m2 = {}
result = True

for i in range(len(s)):
    a = s[i]
    b = t[i]

    if a in m1:
```

```
        if m1[a] != b:
            result = False
            break
    else:
        m1[a] = b

    if b in m2:
        if m2[b] != a:
            result = False
            break
    else:
        m2[b] = a

print(result)
```

---

5. Find Pairs with Given Sum

Using Built-ins

```
arr = [1, 5, 7, -1, 5]
target = 6
seen = set()

for num in arr:
    if target - num in seen:
        print((target - num, num))

    seen.add(num)
```

Without Built-ins

```
arr = [1, 5, 7, -1, 5]
target = 6

for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
        if arr[i] + arr[j] == target:
            print(arr[i], arr[j])
```

Recursion

1. Factorial Using Recursion

Using Built-ins

```
import math

print(math.factorial(5))
```

Without Built-ins

```
def factorial(n):
    if n == 0 or n == 1:
        return 1

    return n * factorial(n - 1)

print(factorial(5))
```

---

2. Fibonacci Series

Using Built-ins

```
n = 10
fib = [0, 1]

for _ in range(2, n):
    fib.append(fib[-1] + fib[-2])

print(fib)
```

Without Built-ins

```
def fib(n):
    if n <= 1:
        return n

    return fib(n - 1) + fib(n - 2)
```

```
for i in range(10):
    print(fib(i), end=' ')
```

---

3. Sum of Digits

Using Built-ins

```
n = 1234
print(sum(int(d) for d in str(n)))
```

Without Built-ins

```
def sum_digits(n):
    if n == 0:
        return 0

    return n % 10 + sum_digits(n // 10)

print(sum_digits(1234))
```

---

4. Reverse a String Recursively

Using Built-ins

```
s = "hello"
print(s[::-1])
```

Without Built-ins

```
def reverse(s):
    if len(s) == 0:
        return s

    return reverse(s[1:]) + s[0]

print(reverse("hello"))
```

---

5. Tower of Hanoi

Recursive Solution

```
def tower(n, source, helper, destination):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    tower(n - 1, source, destination, helper)

    print(f"Move disk {n} from {source} to {destination}")

    tower(n - 1, helper, source, destination)


tower(3, 'A', 'B', 'C')
```

Sorting / Searching

1. Binary Search

Using Built-ins

```
import bisect

arr = [1, 2, 3, 4, 5]
print(bisect.bisect_left(arr, 4))
```

Without Built-ins

```
arr = [1, 2, 3, 4, 5]
target = 4
left = 0
right = len(arr) - 1

while left <= right:
    mid = (left + right) // 2

    if arr[mid] == target:
        print(mid)
```

```
        break

    elif arr[mid] < target:
        left = mid + 1

    else:
        right = mid - 1
```

---

2. Linear Search

Using Built-ins

```
arr = [10, 20, 30, 40]
print(arr.index(30))
```

Without Built-ins

```
arr = [10, 20, 30, 40]
target = 30

for i in range(len(arr)):
    if arr[i] == target:
        print(i)
        break
```

---

3. Bubble Sort

Using Built-ins

```
arr = [5, 1, 4, 2, 8]
print(sorted(arr))
```

Without Built-ins

```
arr = [5, 1, 4, 2, 8]

for i in range(len(arr)):
    for j in range(0, len(arr) - i - 1):
        if arr[j] > arr[j + 1]:
```

```
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print(arr)
```

---

4. Selection Sort

Using Built-ins

```
arr = [64, 25, 12, 22, 11]
print(sorted(arr))
```

Without Built-ins

```
arr = [64, 25, 12, 22, 11]

for i in range(len(arr)):
    min_index = i

    for j in range(i + 1, len(arr)):
        if arr[j] < arr[min_index]:
            min_index = j

    arr[i], arr[min_index] = arr[min_index], arr[i]

print(arr)
```

---

5. Merge Sort

Without Built-ins

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0
```

```
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1

            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1


arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr)
print(arr)
```

---

6. Quick Sort

Without Built-ins

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[0]
    left = []
    right = []

    for num in arr[1:]:
        if num <= pivot:
            left.append(num)
        else:
            right.append(num)
```

```
    return quick_sort(left) + [pivot] + quick_sort(right)


arr = [10, 7, 8, 9, 1, 5]
print(quick_sort(arr))
```

Numbers / Math

1. Prime Number Check

Using Built-ins

```
n = 13

print(all(n % i != 0 for i in range(2, int(n**0.5) + 1)))
```

Without Built-ins

```
n = 13
prime = True

if n <= 1:
    prime = False
else:
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            prime = False
            break

print(prime)
```

---

2. Armstrong Number

Using Built-ins

```
n = 153
power = len(str(n))
```

```
result = sum(int(digit) ** power for digit in str(n))
print(result == n)
```

Without Built-ins

```
n = 153
temp = n
count = 0

while temp > 0:
    count += 1
    temp //= 10

sum_val = 0
temp = n

while temp > 0:
    digit = temp % 10
    sum_val += digit ** count
    temp //= 10

print(sum_val == n)
```

---

3. Fibonacci Number

Using Built-ins

```
n = 10
fib = [0, 1]

for i in range(2, n):
    fib.append(fib[i - 1] + fib[i - 2])

print(fib)
```

Without Built-ins

```
def fibonacci(n):
    a = 0
    b = 1

    for _ in range(n):
```

```
        print(a, end=' ')
        a, b = b, a + b


fibonacci(10)
```

---

4. GCD / LCM

Using Built-ins

```
import math

a = 12
b = 18

print(math.gcd(a, b))
print((a * b) // math.gcd(a, b))
```

Without Built-ins

```
a = 12
b = 18
x = a
y = b

while y != 0:
    x, y = y, x % y

print("GCD:", x)
print("LCM:", (a * b) // x)
```

---

5. Factorial

Using Built-ins

```
import math

print(math.factorial(5))
```

Without Built-ins

```
n = 5
fact = 1

for i in range(1, n + 1):
    fact *= i

print(fact)
```

---

6. Check Perfect Number

Using Built-ins

```
n = 28

result = sum(i for i in range(1, n) if n % i == 0)
print(result == n)
```

Without Built-ins

```
n = 28
sum_val = 0

for i in range(1, n):
    if n % i == 0:
        sum_val += i

print(sum_val == n)
```

OOP / Python-Specific Coding

1. Implement a Class for Bank Account

```
class BankAccount:
    def __init__(self, name, balance=0):
        self.name = name
        self.balance = balance
```

```
    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
        else:
            print("Insufficient balance")

    def display(self):
        print(self.name, self.balance)


acc = BankAccount("John", 1000)
acc.deposit(500)
acc.withdraw(300)
acc.display()
```

---

2. Use Decorators

```
def decorator(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")

    return wrapper


@decorator
def hello():
    print("Hello")


hello()
```

---

3. Implement Generator for Fibonacci

```
def fibonacci(n):
    a, b = 0, 1
```

```
    for _ in range(n):
        yield a
        a, b = b, a + b


for num in fibonacci(10):
    print(num)
```

---

4. Flatten a Nested List

Using Built-ins

```
nested = [[1, 2], [3, 4], [5]]

flat = [item for sublist in nested for item in sublist]
print(flat)
```

Without Built-ins

```
def flatten(arr):
    result = []

    for item in arr:
        if type(item) == list:
            result.extend(flatten(item))
        else:
            result.append(item)

    return result


nested = [1, [2, [3, 4]], 5]
print(flatten(nested))
```

---

5. Implement Stack Using List

```
stack = []

stack.append(10)
stack.append(20)
```

```
stack.append(30)

print(stack.pop())
print(stack)
```

---

6. Implement Queue Using List / Deque

Using List

```
queue = []

queue.append(10)
queue.append(20)
queue.append(30)

print(queue.pop(0))
print(queue)
```

Using Deque

```
from collections import deque

queue = deque()

queue.append(10)
queue.append(20)
queue.append(30)

print(queue.popleft())
print(queue)
```